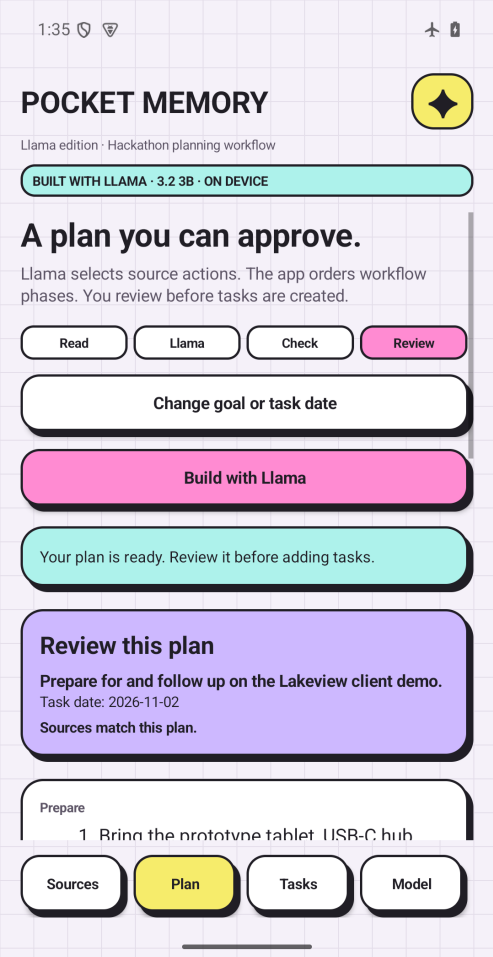
META GLOBAL AI DEVELOPER HACKATHON 2026

Pocket Memory Built with Llama

Saved instructions.
Reviewed action.



A local planner.
A bounded task engine.

Meta Llama 3.2 3B selects relevant source actions. Application rules order workflow phases. The user inspects the evidence and approves a local task batch.

Changed sources block old proposals. Repeated apply avoids duplicates. Guarded undo protects completed or changed work.

Llama hackathon edition
Indie Developer Track
Build for Your World
Prepared 1 November 2026

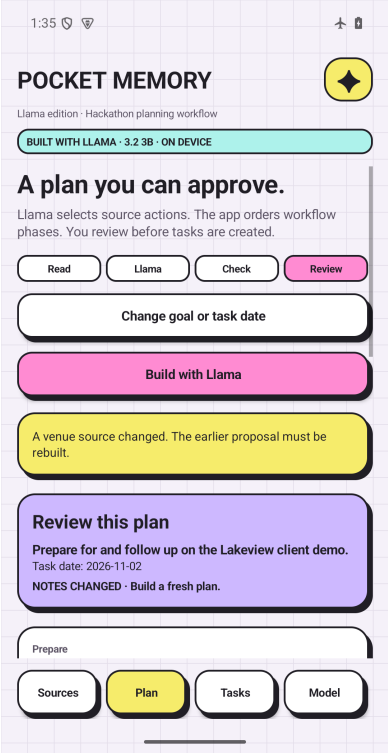
Manthan Dhole | Solo developer

Actual app captures, fictional data and real Meta inference. The narrated preview is edited and makes no speed or physical-phone performance claim.

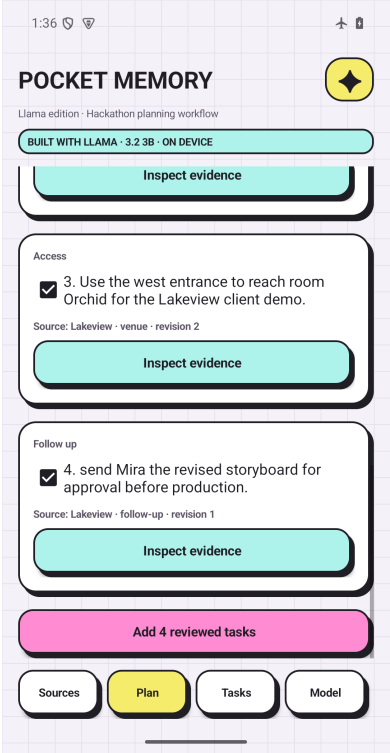
The client-day plan

Five fictional notes contain arrival instructions, equipment, venue, follow-up and a separate party. The goal is to prepare for and follow up on the Lakeview client demo. Llama selects relevant IDs; the application copies exact source action text.

Source	Relevant detail
Arrival	2 November 2026; arrive 09:45 for the 10:00 demo.
Equipment	Bring the prototype tablet, USB-C hub and printed storyboard.
Venue change	East entrance / Cedar becomes west entrance / Orchid.
Follow-up	Send Mira the revised storyboard for approval.
Distractor	Blue balloons belong to a different event and are excluded.



An edited source blocks the old batch.



Rebuild, inspect and explicitly approve.

A model with bounded authority

Source snapshot -> candidate actions -> Meta Llama selects IDs -> validation -> human review -> atomic task batch -> guarded undo.

Boundary	Implementation
Sources	Immutable note list and a hash of that same snapshot. Up to 50 notes.
Candidates	Goal keyword ranking; up to 8 notes and 16 explicit action sentences.
Inference	Meta Llama 3.2 3B Instruct Q4_K_M through a new JNI adapter and public llama.cpp.
Output	Grammar-constrained JSON: one steps array, allowed integer IDs. Java rejects unknown, duplicate, fractional or extra fields. Up to 6 actions.
Review	Exact source text and source revisions remain inspectable. Llama cannot create tasks itself.
Apply	Source freshness, deduplication, task writes and batch links share one SQLite transaction.
Undo	Check every created task first; preserve reused, completed or changed work.
Recovery	Job-scoped cancellation, bounded inference, persisted proposals/tasks and no automatic re-execution on restart.

Pinned model identity

Upstream: meta-llama/Llama-3.2-3B-Instruct
GGUF: bartowski/Llama-3.2-3B-Instruct-GGUF
Revision: 5ab33fa94d1d04e903623ae72c95d1696f09f9e8
Size: 2,019,377,696 bytes

SHA-256: 6c1a2b41161032677be168d354123594c0e6e67d2b9227c84f296ad037c728ff

Built with Llama. Model weights are imported separately after size/hash verification. The model is governed by its Llama 3.2 Community License and Acceptable Use Policy.

Inspectable verification

Build and Android lint passed. The real ARM64 Android 15 emulator walkthrough runs in airplane mode. An independent restart test verifies persisted task identities and batch links.

Test	Observed behavior
Meta inference	Actual Llama selects the relevant client actions and excludes the separate party. Raw ID output is recorded.
Ordering	Application-owned phase rules put preparation before arrival, venue access and explicit follow-up in the verified proposal.
Output validation	Unknown IDs, duplicate IDs, fractions, extra fields and prose are refused.
Human review	Planning creates zero tasks; the real review control commits the chosen batch.
Source changes	An outdated proposal rejects the whole transaction and creates zero partial tasks. A new Llama run uses Orchid.
Idempotency	Repeating apply does not duplicate the batch.
Undo	A completed created task blocks all deletion. Unchanged created tasks can be removed; reapply requires explicit review.
Restart	Tasks and links survive actual process restart without automatic inference or duplicate execution.

Limits of the evidence

Structural validation is not proof of relevance or correct phase grouping. Llama may omit a useful action or select a wrong one. Candidate extraction is bounded and expects explicit English action sentences. Plans use one user-selected date and do not infer deadlines or send messages. Physical-phone performance, battery use and broader quality/accessibility evaluation remain pending.

Evidence: docs/evidence/test-meta-llama.txt, test-planner-restart.txt, validation.json and media-provenance.json. No production accuracy percentage, latency benchmark or user-traffic claim is made.

Hackathon contributions.

Built for this entry	Pre-existing foundation
Focused Android planner screens, source snapshots, candidate extraction and new constrained JNI adapter.	Pocket Memory product concept and visual palette; public llama.cpp inference runtime.
Proposal validation, review, stale-source gates, atomic task execution, deduplication and guarded undo.	Meta Llama 3.2 3B open weights and third-party GGUF quantization.
Offline workflow/restart verification, screenshots, animation, narrated demo and documentation.	Android, SQLite and the standard Android build toolchain.

Android preview

Pocket Memory Llama edition: com.pocketmemory.llama, version 0.6.0. The evaluation APK uses Android debug signing and stores sources, proposals and reviewed task batches locally.

Try the preview

Install the release APK on ARM64 Android 13+. In Model, review Llama terms, download the pinned GGUF in your browser, then import it. In Sources, load the fictional example. Build with Llama, inspect the evidence and review before applying. Allow 2.1 GB for model storage plus runtime space; RAM guidance is estimated.

Useful next applications

Community workshops, household projects and presentation preparation can use the same explicit-instruction pattern. These are intended extensions; only the client-demo scenario has recorded model verification.

Project and official references

Repository: github.com/MANTHAN137/pocket-memory-llama
Demo / APK / checksum: GitHub release v0.6.0-plans
New source: MIT; model and vendor terms remain separate.
Event: cerebralvalley.ai/e/meta-ai-developer-hackathon
Guide: event URL + /details; waiver: event URL + /waiver
Deadline: 5 November 2026, 7:30 AM IST

The demo highlights new hackathon work. No hosted Meta API, Muse model, API credits or bonus-prize eligibility is claimed. Final organizer permission acceptance remains with the participant.